Reto I: Clasificación de Opiniones sobre Hoteles en Chicago

Autor: Mario Núñez Hernández

16 de octubre de 2026

This text has been made in LaTeX

1. Introducción al Problema

El objetivo de este reto es construir un sistema de Machine Learning (ML) capaz de clasificar opiniones sobre hoteles en Chicago como **verídicas** (truthful) o **engañosas/spam** (deceptive). Este es un problema clásico de Procesamiento de Lenguaje Natural (NLP) y clasificación binaria.

A continuación, desglosaremos cada punto del reto abordando la teoría subyacente, la aplicación al caso práctico y las implementaciones de código necesarias.

2. 1. Variable Objetivo, Histograma y Análisis de Textos

2.1. Teoría

En un problema de clasificación supervisada, la **variable objetivo** (target) es la etiqueta que queremos predecir. En el dataset proporcionado, la variable objetivo es la columna `deceptive`, que contiene dos clases: `truthful` y `deceptive`.

El cálculo de un **histograma** (o gráfico de barras para variables categóricas) permite analizar la distribución de clases. Un conjunto de datos desbalanceado requeriría técnicas especiales de muestreo o funciones de pérdida ponderadas.

Patrones textuales: En la literatura psicológica y computacional, las opiniones falsas a menudo abusan de pronombres en primera persona ("yo", "nosotros") para compensar la falta de experiencia real, y carecen de detalles espaciales concretos sobre la habitación o el entorno del hotel.

2.2. Implementación en Python

```python
import pandas as pd
import matplotlib.pyplot as plt

# Cargar los datos
url = "https://raw.githubusercontent.com/AnIsAsPe/
    LogisticRegression_SpamOpinion/master/Datos/deceptive-opinion.csv"
df = pd.read_csv(url)

# Identificar la variable objetivo
print(df['deceptive'].value_counts())

# Generar un histograma / grafico de barras
df['deceptive'].value_counts().plot(kind='bar', color=['skyblue', '
    salmon'])
plt.title('Distribucion de la Variable Objetivo')
plt.xlabel('Clase')
plt.ylabel('Frecuencia')
plt.show()

# Analisis de ejemplos
```

```
print("Ejemplo Truthful:\n", df[df['deceptive'] == 'truthful']['text'].
    iloc[0])
print("\nEjemplo Deceptive:\n", df[df['deceptive'] == 'deceptive']['
    text'].iloc[0])
```

3. 2. Nubes de Palabras (Word Clouds)

3.1. Teoría

Una **nube de palabras** es una representación visual donde el tamaño de cada palabra indica su frecuencia o importancia en el texto. Para que sea efectiva, se deben eliminar las *stop words* (palabras comunes sin significado inherente como "el", "la", "y"). Al separar el texto por clases, las nubes revelan léxicos distintivos: por ejemplo, las opiniones verídicas pueden centrarse en aspectos tangibles del hotel ("baño", "ubicación"), mientras que las falsas pueden tener verbos exagerados o centrarse en la persona ("esposo", "viaje empresarial").

3.2. Implementación en Python

```
from wordcloud import WordCloud

# Separar el texto por clases
text_truthful = " ".join(df[df['deceptive'] == 'truthful']['text'].
    tolist())
text_deceptive = " ".join(df[df['deceptive'] == 'deceptive']['text'].
    tolist())

# Generar nubes de palabras
wc_t = WordCloud(width=800, height=400, background_color='white').
    generate(text_truthful)
wc_d = WordCloud(width=800, height=400, background_color='black').
    generate(text_deceptive)

# (El codigo para graficar plt.imshow(wc_t) se omitiria por brevedad)
```

4. 3. Bolsa de Palabras (Bag of Words) y Vectorización TF-IDF

4.1. Teoría

Los algoritmos de Machine Learning no pueden procesar texto crudo; requieren vectores numéricos.

- **Bag of Words (BoW):** Cuenta la frecuencia de cada palabra en cada documento. Ignora el orden y la semántica, pero es una buena línea base empírica. El dataframe supervisado resultante tendrá tantas columnas como palabras únicas en el vocabulario, y las filas representarán las reseñas.

- **TF-IDF (Term Frequency - Inverse Document Frequency):** Es una mejora matemática de BoW. Penaliza las palabras que aparecen frecuentemente en todos los documentos (y por tanto no son discriminativas) y otorga mayor peso a palabras frecuentes en un documento específico pero raras en el corpus global. La fórmula base es:

$$\text{TF-IDF}(t, d) = tf_{t,d} \times \log\left(\frac{N}{df_t}\right)$$

4.2. Implementación en Python

```python
from sklearn.feature_extraction.text import CountVectorizer,
    TfidfVectorizer

# 1. Bag of Words
bow_vectorizer = CountVectorizer(stop_words='english', max_features
    =1000)
X_bow = bow_vectorizer.fit_transform(df['text'])
df_bow = pd.DataFrame(X_bow.toarray(), columns=bow_vectorizer.
    get_feature_names_out())
df_bow['target_class'] = df['deceptive'].values # Dataframe supervisado

# 2. Vectorizacion con TF-IDF
tfidf_vectorizer = TfidfVectorizer(stop_words='english', max_features
    =1000)
X_tfidf = tfidf_vectorizer.fit_transform(df['text'])
# X_tfidf se utilizara para el modelo de ML
```

5. 4. Entrenamiento del Modelo Tradicional de Machine Learning

5.1. Teoría

Para la clasificación binaria de textos dispersos y de alta dimensionalidad (como TF-IDF), la **Regresión Logística** o las **Máquinas de Vectores de Soporte (SVM)** son los modelos más eficientes. La regresión logística modela la probabilidad de que una reseña sea falsa mediante la función sigmoide aplicada a una combinación lineal de los valores TF-IDF.

5.2. Implementación en Python

```python
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

# Convertir etiquetas a binario
y = df['deceptive'].map({'truthful': 0, 'deceptive': 1})

# Division de datos
```

```
X_train, X_test, y_train, y_test = train_test_split(X_tfidf, y,
    test_size=0.2, random_state=42)

# Entrenar modelo
model = LogisticRegression()
model.fit(X_train, y_train)

# Evaluacion
y_pred = model.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

6. 5. Enfoque con Modelos de Lenguaje (LLMs)

6.1. Teoría

A diferencia del enfoque tradicional basado en representaciones léxicas (TF-IDF), los LLMs (como GPT-4, Gemini, Claude) utilizan arquitecturas Transformer que comprenden el contexto profundo y la semántica de la oración completa. Pueden realizar la tarea mediante *Zero-shot prompting* (pidiéndole directamente que clasifique sin entrenamiento previo) o mediante *Fine-tuning*.

6.2. Implementación alternativa usando API

Para implementar una solución usando un LLM a través de una API (por ejemplo, OpenAI API):

```
import openai

openai.api_key = "TU_API_KEY"

def clasificar_opinion_llm(texto):
    prompt = f"""
    Actua como un experto en deteccion de fraudes en textos.
    Clasifica la siguiente opinion sobre un hotel en Chicago como '
    truthful' (veridica) o 'deceptive' (falsa/spam).
    Responde unicamente con una de las dos palabras.
    Opinion: "{texto}"
    """

    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.0
    )
    return response.choices[0].message.content.strip().lower()

# Ejemplo de uso
# prediccion = clasificar_opinion_llm(df['text'].iloc[0])
```

7. 6. Análisis Comparativo: ML Tradicional vs LLMs

Para seleccionar la mejor solución, evaluamos cuatro dimensiones críticas dictadas en el reto:

- **Métricas y Desempeño (Matriz de Confusión):** Los modelos tradicionales como Logistic Regression logran un rendimiento muy sólido en este dataset específico (accuracy superior al 85 %) porque el spam de opiniones tiene firmas léxicas claras. Sin embargo, los LLMs suelen lograr precisión de estado del arte (SOTA) especialmente en casos ambiguos donde el sarcasmo o el contexto oculto es clave, aunque pueden ser propensos a alucinaciones o respuestas fuera de formato en modo zero-shot.
- **Explicabilidad del Modelo:**
  - *ML Tradicional:* Altamente explicable. En la Regresión Logística, podemos inspeccionar los coeficientes (pesos) de cada palabra. Si la palabra τo"tiene un peso positivo alto, sabemos matemáticamente por qué el modelo clasificó la reseña como engañosa.
  - *LLMs:* Operan como çajas negras". Aunque podemos pedirle al modelo que explique su razonamiento (Chain of Thought), no hay una garantía matemática que conecte sus representaciones internas con la justificación textual.
- **Tiempo de Inferencia:**
  - *ML Tradicional:* Inferencia casi instantánea (microsegundos por registro). Escala fácilmente para procesar millones de reseñas localmente.
  - *LLMs:* La inferencia de red introducida por las APIs añade latencia (cientos de milisegundos a segundos por registro). Restringe las aplicaciones en tiempo real estricto y tiene límites de tasa (rate limits).
- **Costos Computacionales y Monetarios:**
  - *ML Tradicional:* El costo computacional se da durante el entrenamiento (muy bajo para TF-IDF). El despliegue y la inferencia son gratuitos (costo de infraestructura propio mínimo).
  - *LLMs:* Cero costo de entrenamiento (en zero-shot), pero altísimo costo por inferencia. Procesar datasets masivos a $0.0015 por cada 1K tokens se vuelve un gasto operativo significativo y continuo.

Conclusión Estratégica: Para este escenario, se recomienda implementar un **modelo híbrido o cascada**. Utilizar el modelo ML tradicional (TF-IDF + Regresión Logística) como el sistema primario debido a su bajo costo, alta velocidad y explicabilidad. El LLM se reserva exclusivamente para los casos

donde el modelo tradicional muestra baja confianza (probabilidades de predicción cercanas al umbral del 0.5), optimizando así la relación entre rendimiento y costo.

Repositorio de solución código

Puede consultar el código fuente de ambas implementaciones y más detalles en el siguiente enlace:
`https://github.com/MarioNunezH/NLP2602-Primer-Tema/tree/main`

Referencias

[1] Ott, M., Choi, Y., Cardie, C., & Hancock, J. T. (2011). *Finding Deceptive Opinion Spam by Any Stretch of the Imagination*. En Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics.

[2] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. (Para conceptos de TF-IDF y Bag of Words).

[3] Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed. draft). (Para arquitecturas de LLMs y clasificación de texto).